

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем

Факультет автоматики и вычислительной техники

Кафедра электронных вычислительных машин

Отчет по лабораторной работе №2

по дисциплине

«Объектно-ориентированное программирование»

Выполнил студент гр. ИВТб-2301-05-00

_____/Черкасов А. А./

Руководитель преподаватель

_____/Шмакова Н. А./

Киров 2026

Цель работы

Цель работы: изучить и освоить принципы объектно-ориентированного программирования, включая инкапсуляцию, наследование и полиморфизм, путем создания иерархии классов в выбранной предметной области (управление гоночными командами), взаимодействия с базой данных и графическим веб-интерфейсом.

Задание

На основании согласованной в первой лабораторной работе предметной области разработать приложение с графическим интерфейсом пользователя, используя классы и особенности C#. Сведения об объектах предметной области необходимо хранить в БД.

Решение

Предметная область описывает управление гоночными командами трех различных серий (Formula 1, Formula 2, Formula E). Система позволяет хранить информацию о командах и их гонщиках, учитывать их специфические характеристики, проводить симуляцию гонок с начислением очков гонщикам и командам по правилам конкретной серии, выполнять сравнение команд по очкам через перегруженные операторы и выводить информацию о них в удобном виде.

Программа содержит классы «FormulaTeam», представляющий базовый абстрактный класс команды, «F1Team», наследованный от «FormulaTeam» и представляющий класс команды Formula 1, «F2Team», наследованный от «FormulaTeam» и представляющий класс команды Formula 2, «FormulaETeam», наследованный от «FormulaTeam» и представляющий класс команды Formula E, а также вспомогательный класс «FormulaDriver» для хранения информации о гонщиках.

Базовый класс «FormulaTeam» содержит:

– Свойства:

- public int Id { get; set; } – свойство, уникальный идентификатор;
- public string TeamName { get; set; } – свойство, название команды;

- public string PrincipalName { get; set; } – свойство, имя руководителя команды;
- public string Headquarters { get; set; } – свойство, штаб-квартира команды;
- public string? TeamColor { get; set; } – свойство, цвет команды;
- public int ChampionshipPoints { get; set; } – свойство, очки чемпионата;
- public int RaceWins { get; set; } – свойство, количество побед;
- public int Podiums { get; set; } – свойство, количество подиумов;
- public List<FormulaDriver> Drivers { get; set; } – свойство, список гонщиков;
- Конструкторы:
 - protected FormulaTeam() – защищенный конструктор по умолчанию;
 - protected FormulaTeam(string name, string principal, string hq) – защищенный конструктор, принимает название, имя руководителя и штаб-квартиру;
 - protected FormulaTeam(string name, string principal, string hq, int points, int wins, int podiums) – защищенный конструктор, принимает все основные параметры;
- Абстрактные методы:
 - public abstract string GetSeriesName() – публичный абстрактный метод, возвращающий название серии;
 - public abstract int CalculatePoints(int position) – публичный абстрактный метод, возвращающий очки за занятую позицию;
- Перегруженные операторы:
 - public static bool operator >(FormulaTeam a, FormulaTeam b) – оператор больше, сравнивает очки команд;
 - public static bool operator <(FormulaTeam a, FormulaTeam b) – оператор меньше, сравнивает очки команд;
- Собственные методы:

- `public void AddPoints(int pts)` – публичный метод, добавляет очки команде;
- `public void AddWin()` – публичный метод, увеличивает количество побед и подиумов;
- `public void AddPodium()` – публичный метод, увеличивает количество подиумов;
- `public double GetWinRate(int totalRaces)` – публичный метод, рассчитывает процент побед;
- `public virtual string GetSummary()` – публичный виртуальный метод, возвращает сводку по команде.

Класс «F1Team», наследованный от класса «FormulaTeam» содержит:

– Собственные свойства:

- `public string PowerUnit { get; set; }` – свойство, тип силовой установки;
- `public double BudgetCapMln { get; set; }` – свойство, лимит бюджета в миллионах;
- `public int ConstructorPos { get; set; }` – свойство, позиция в кубке конструкторов;

– Конструкторы:

- `public F1Team() : base()` – публичный конструктор по умолчанию;
- `public F1Team(string name, string principal, string hq, string powerUnit) : base(name, principal, hq)` – конструктор, принимающий основные параметры;
- `public F1Team(string name, string principal, string hq, string powerUnit, double budget, int ctorPos, int points, int wins, int podiums) : base(name, principal, hq, points, wins, podiums)` – конструктор, принимающий все параметры;

– Переопределенные методы:

- `public override string GetSeriesName()` – возвращает «Formula 1»;
- `public override int CalculatePoints(int position)` – возвращает очки по системе F1 (25-18-15-12-10-8-6-4-2-1);

- `public override string GetSummary()` – возвращает сводку с учетом силовой установки и бюджета;
- Собственные методы:
 - `public double CalculateBudgetAllocation(double percentage)` – метод расчета выделения бюджета;
 - `public string GetPowerUnitStatus()` – возвращает статус силовой установки (Dominant, Competitive, Development).

Класс «F2Team», наследованный от класса «FormulaTeam» содержит:

- Собственные свойства:
 - `public string ChassisModel { get; set; }` – свойство, модель шасси;
 - `public int F1Graduates { get; set; }` – свойство, число выпускников, перешедших в F1;
 - `public bool IsFeederSeries { get; set; }` – свойство, является ли молодежной серией;
- Конструкторы:
 - `public F2Team() : base()` – конструктор по умолчанию;
 - `public F2Team(string name, string principal, string hq, string chassis) : base(name, principal, hq)` – конструктор, принимающий основные параметры;
 - `public F2Team(string name, string principal, string hq, string chassis, int graduates, bool feeder, int points, int wins, int podiums) : base(name, principal, hq, points, wins, podiums)` – конструктор, принимающий все параметры;
- Переопределенные методы:
 - `public override string GetSeriesName()` – возвращает «Formula 2»;
 - `public override int CalculatePoints(int position)` – возвращает очки по системе F2 (15-12-10-8-6-5-4-3-2-1);
 - `public override string GetSummary()` – возвращает сводку с учетом модели шасси и числа выпускников;
 - `public string GetFeederStatus()` – возвращает статус молодежной

команды.

Класс «FormulaETeam», наследованный от класса «FormulaTeam» содержит:

– Собственные свойства:

- public int SustainabilityScore { get; set; } – свойство, рейтинг экологичности;
- public double BatteryCapacityKwh { get; set; } – свойство, емкость батареи;
- public string EnergyPartner { get; set; } – свойство, энергетический партнер;

– Конструкторы:

- public FormulaETeam() : base() – конструктор по умолчанию;
- public FormulaETeam(string name, string principal, string hq, string energyPartner) : base(name, principal, hq) – конструктор с основными параметрами;
- public FormulaETeam(string name, string principal, string hq, string energyPartner, int sustainScore, double batteryKwh, int points, int wins, int podiums) : base(name, principal, hq, points, wins, podiums) – конструктор со всеми параметрами;

– Переопределенные методы:

- public override string GetSeriesName() – возвращает «Formula E»;
- public override int CalculatePoints(int position) – возвращает очки по системе Formula E;
- public override string GetSummary() – возвращает сводку с учетом экологичности и емкости батареи;

– Собственные методы:

- public string GetBatteryEfficiencyRating() – возвращает рейтинг эффективности батареи.

Класс «FormulaDriver» представляет гонщика и содержит:

– Свойства:

- public int Id { get; set; } – свойство, уникальный идентификатор;

- `public int TeamId { get; set; }` – свойство, идентификатор команды (внешний ключ);
- `public string FirstName { get; set; }` – свойство, имя гонщика;
- `public string LastName { get; set; }` – свойство, фамилия гонщика;
- `public int RacingNumber { get; set; }` – свойство, гоночный номер;
- `public int Points { get; set; }` – свойство, набранные очки;
- Конструкторы:
 - `public FormulaDriver()` – публичный конструктор по умолчанию;
 - `public FormulaDriver(string firstName, string lastName, int number, int points = 0)` – параметризованный конструктор;
- Собственные методы:
 - `public void AddPoints(int pts)` – увеличивает количество набранных очков;
- Вычисляемые свойства:
 - `public string FullName { get; }` – возвращает полное имя гонщика;
 - `public string DisplayString { get; }` – возвращает строку отображения пилота.

Диаграмма классов сущностей представлена на рисунке 1.

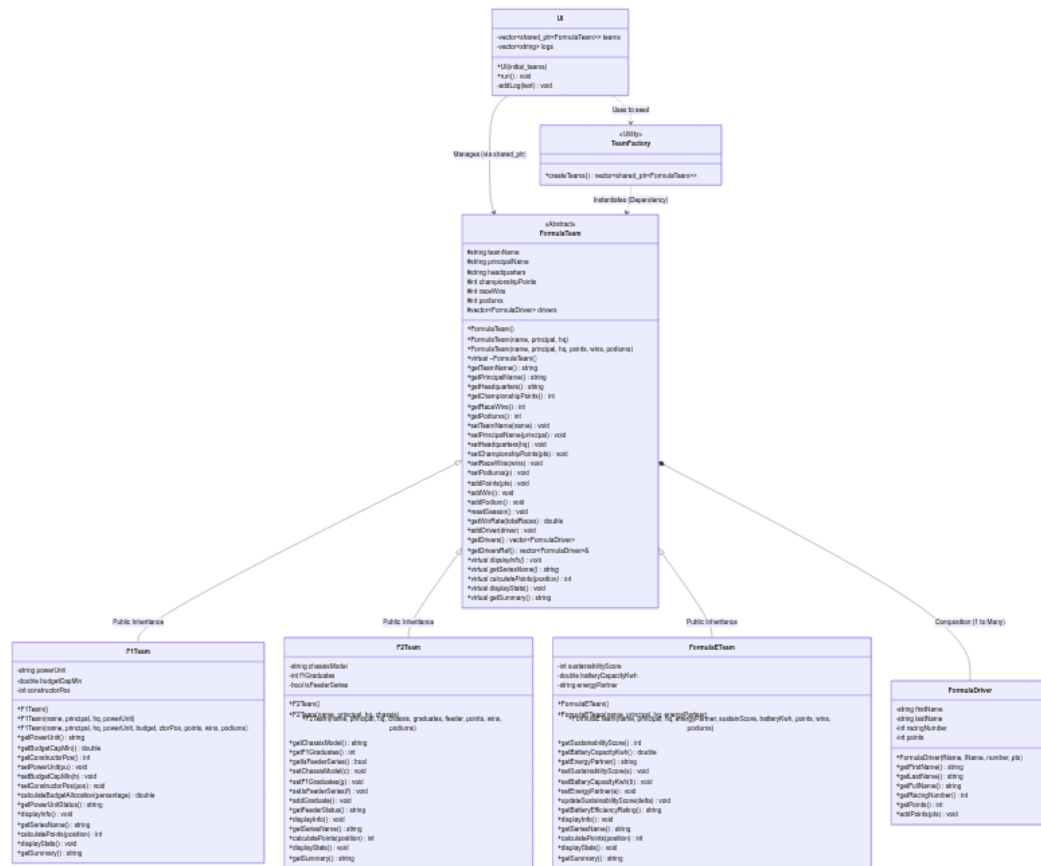


Рисунок 1 – Диаграмма классов сущностей

Разработаны серверное приложение на бэкенде в виде REST API на C# и клиентское приложение в виде веб-интерфейса (React). Серверное приложение содержит подключение к базе данных PostgreSQL, конфигурацию моделей с использованием TPH и маршруты API.

Подключение к базе данных происходит с помощью следующей строки подключения в методе OnConfiguring класса RacingDbContext:

```
optionsBuilder.UseNpgsql(
    "Host=localhost;Database=postgres;" +
    "Username=postgres;Password=postgres");
```

И сам класс контекста:

```
public class RacingDbContext : DbContext
{
    public DbSet<FormulaTeam> Teams { get; set; } = null!;
    public DbSet<FormulaDriver> Drivers { get; set; } = null!;
```

```

protected override void OnConfiguring(
    DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseNpgsql(
        "Host=localhost;Database=postgres;" +
        "Username=postgres;Password=postgres");
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Настройка TPH и связей
    base.OnModelCreating(modelBuilder);
}
}

```

Конфигурация сущности FormulaTeam с использованием стратегии Table-per-Hierarchy (TPH) содержит настройку дискриминатора по полю series_id для деления на F1, F2 и Formula E:

```

modelBuilder.Entity<FormulaTeam>()
    .ToTable("racing_teams")
    .HasDiscriminator<long>("series_id")
    .HasValue<F1Team>(1)
    .HasValue<F2Team>(2)
    .HasValue<FormulaETeam>(3);

```

Следующим шагом является маршрутизация API-запросов от клиентов. Обработчик ProcessRequest в классе Program выполняет роль маршрутизатора API.

Содержит следующие методы:

- GET /api/teams – возвращает список всех команд с их гонщиками, отсортированных по очкам;
- POST /api/teams – создает новую команду;
- PUT /api/teams/{id} – обновляет существующую команду;
- DELETE /api/teams/{id} – удаляет команду и каскадно удаляет привязанных к ней гонщиков;
- POST /api/teams/{id}/simulate-race – симулирует гонку для команды, полиморфно вычисляя очки по занятому месту и сохраняя изменения в базе данных:

```
int points = team.CalculatePoints(position);
driver.AddPoints(points);
team.AddPoints(points);
```

- GET /api/drivers – возвращает список всех гонщиков;
- POST /api/drivers – создает запись о гонщике;
- PUT /api/drivers/{id} – обновляет запись о гонщике;
- DELETE /api/drivers/{id} – удаляет запись о гонщике по идентификатору;
- GET /api/teams/compare – метод сравнения двух команд по очкам с помощью перегруженных операторов:

```
public static bool operator >(FormulaTeam a, FormulaTeam b)
    => a.ChampionshipPoints > b.ChampionshipPoints;
```

Диаграмма классов контроллеров и конфигураций сущностей представлены на рисунке 2.

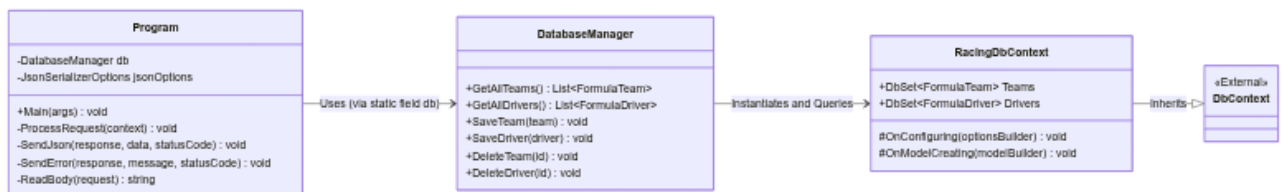


Рисунок 2 – Диаграмма классов базы данных и API

Также разработан веб-интерфейс для взаимодействия с WEB API, экранные формы которого представлены на рисунках 3, 4, 5 и 6.

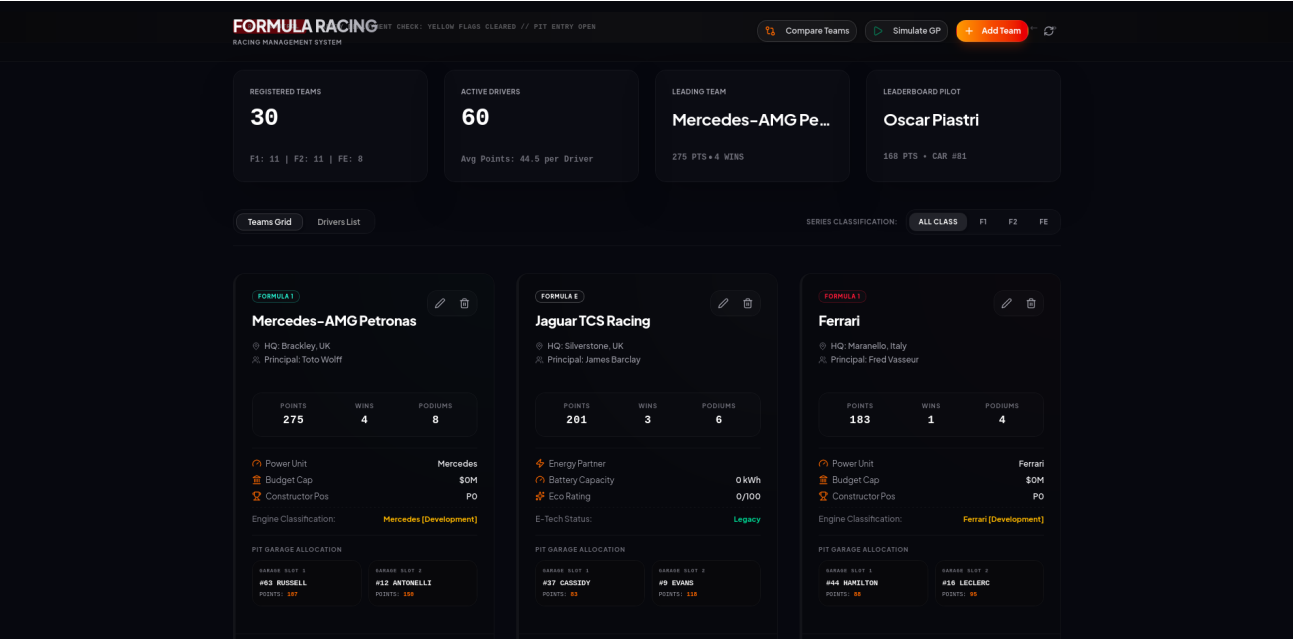


Рисунок 3 – Экранная форма вкладки команд (standings)

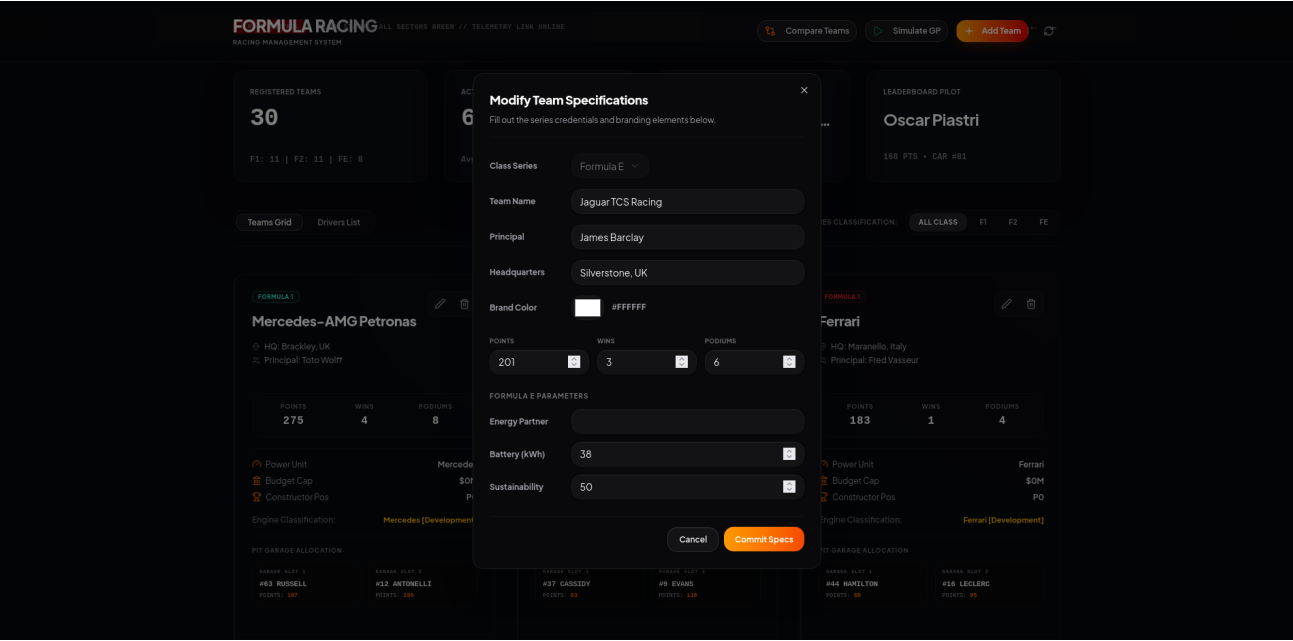


Рисунок 4 – Экранная форма вкладки гонщиков

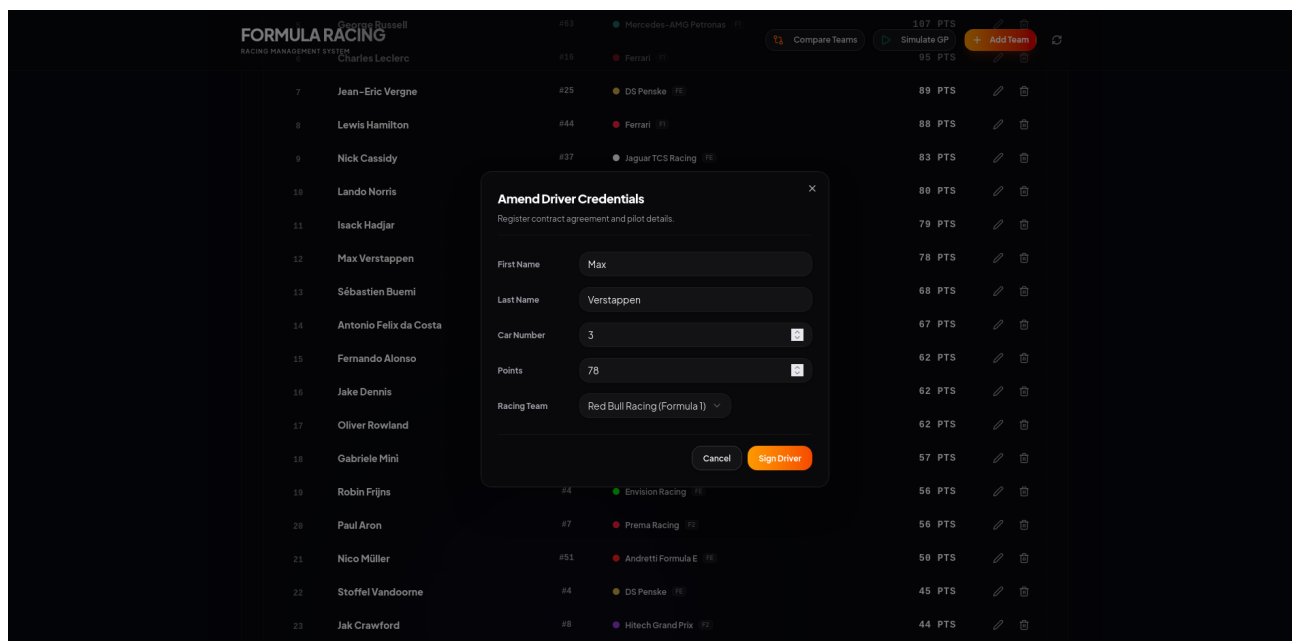


Рисунок 5 – Экранная форма создания и редактирования команд

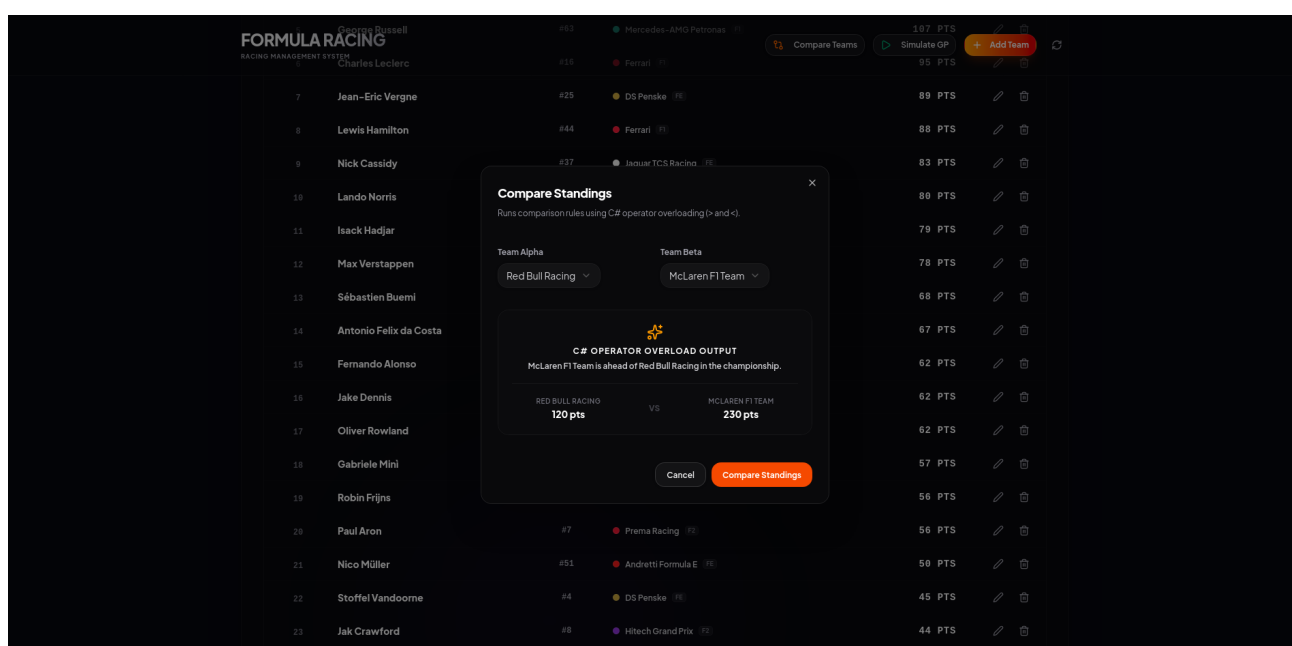


Рисунок 6 – Экранная форма создания и редактирования гонщиков

Вывод

В ходе выполнения лабораторной работы была разработана и реализована на языке C# иерархия классов, моделирующая управление гоночными командами различных серий. Изучены основные принципы объектно-ориентированного

программирования: инкапсуляция, наследование, полиморфизм. Реализованы механизмы работы с реляционной базой данных PostgreSQL через Entity Framework Core с использованием стратегии Table-per-Hierarchy (TPH). Разработано веб-приложение на базе React и бэкенда на C# REST API, демонстрирующее корректное применение данных принципов.

Приложение A1. Базовый класс FormulaTeam.cs

```
using System;
using System.Collections.Generic;
using System.Text.Json.Serialization;

namespace RacingSystem.Core
{
    [JsonPolymorphic(TypeDiscriminatorPropertyName = "type")]
    [JsonDerivedType(typeof(F1Team), "f1")]
    [JsonDerivedType(typeof(F2Team), "f2")]
    [JsonDerivedType(typeof(FormulaETeam), "fe")]
    public abstract class FormulaTeam
    {
        // Enforce encapsulation with Properties
        public int Id { get; set; }
        public string TeamName { get; set; } = "Unknown";
        public string PrincipalName { get; set; } = "Unknown";
        public string Headquarters { get; set; } = "Unknown";
        [JsonPropertyName("team_color")]
        public string? TeamColor { get; set; }
        public int ChampionshipPoints { get; set; }
        public int RaceWins { get; set; }
        public int Podiums { get; set; }
        public List<FormulaDriver> Drivers { get; set; } = new
            ↪ List<FormulaDriver>();

        protected FormulaTeam() { }

        protected FormulaTeam(string name, string principal, string hq)
        {
            TeamName = name;
            PrincipalName = principal;
            Headquarters = hq;
        }

        protected FormulaTeam(string name, string principal, string hq, int
            ↪ points, int wins, int podiums)
            : this(name, principal, hq)
        {

```

```

        ChampionshipPoints = points;
        RaceWins = wins;
        Podiums = podiums;
    }

    // Logic methods from Lab 1
    public void AddPoints(int pts)
    {
        if (pts > 0) ChampionshipPoints += pts;
    }

    public void AddWin()
    {
        RaceWins++;
        Podiums++;
    }

    public void AddPodium() => Podiums++;

    public double GetWinRate(int totalRaces)
    {
        if (totalRaces <= 0) return 0.0;
        return (double)RaceWins / totalRaces * 100.0;
    }

    // Abstract methods (requirement from Lab 1)
    // Logic methods
    public abstract string GetSeriesName();
    public abstract int CalculatePoints(int position);

    public virtual string GetSummary()
    {
        return $"[{GetSeriesName()}] {TeamName} | Pts: {ChampionshipPoints} |
        ↳ Wins: {RaceWins}";
    }

    // Properties for UI binding
    public string SeriesName => GetSeriesName();
    public string Summary => GetSummary();

```

```

[JsonPropertyName("visible_color")]
public string VisibleColor => GetVisibleTeamColor(TeamColor);

private static string GetVisibleTeamColor(string? hex)
{
    if (string.IsNullOrEmpty(hex)) return "#3b82f6";

    string cleanHex = hex.Replace("#", "").Trim();
    if (cleanHex.Equals("000000", StringComparison.OrdinalIgnoreCase) ||
        cleanHex.Equals("000", StringComparison.OrdinalIgnoreCase) ||
        cleanHex.Equals("1E293B", StringComparison.OrdinalIgnoreCase) ||
        cleanHex.Equals("18181b", StringComparison.OrdinalIgnoreCase))
    {
        return "#a1a1aa";
    }

    if (cleanHex.Length == 3)
    {
        cleanHex = $"{cleanHex[0]}{cleanHex[0]}" +
            $"{cleanHex[1]}{cleanHex[1]}" +
            $"{cleanHex[2]}{cleanHex[2]}";
    }

    if (cleanHex.Length != 6) return hex;

    try
    {
        int rVal = Convert.ToInt32(cleanHex.Substring(0, 2), 16);
        int gVal = Convert.ToInt32(cleanHex.Substring(2, 2), 16);
        int bVal = Convert.ToInt32(cleanHex.Substring(4, 2), 16);

        double r = rVal / 255.0;
        double g = gVal / 255.0;
        double b = bVal / 255.0;

        double luminance = 0.2126 * r + 0.7152 * g + 0.0722 * b;

        if (luminance < 0.35)
        {
            double max = Math.Max(r, Math.Max(g, b));

```

```

double min = Math.Min(r, Math.Min(g, b));
double h = 0, s = 0, l = (max + min) / 2.0;

if (max != min)
{
    double d = max - min;
    s = l > 0.5 ? d / (2.0 - max - min) : d / (max + min);
    if (max == r)
    {
        h = (g - b) / d + (g < b ? 6.0 : 0.0);
    }
    else if (max == g)
    {
        h = (b - r) / d + 2.0;
    }
    else if (max == b)
    {
        h = (r - g) / d + 4.0;
    }
    h /= 6.0;
}

l = Math.Max(l, 0.55);
s = Math.Max(s, 0.65);

double Hue2Rgb(double pVal, double qVal, double tVal)
{
    if (tVal < 0) tVal += 1.0;
    if (tVal > 1.0) tVal -= 1.0;
    if (tVal < 1.0 / 6.0) return pVal + (qVal - pVal) * 6.0 *
        ↪ tVal;
    if (tVal < 1.0 / 2.0) return qVal;
    if (tVal < 2.0 / 3.0) return pVal + (qVal - pVal) * (2.0 /
        ↪ 3.0 - tVal) * 6.0;
    return pVal;
}

double q = l < 0.5 ? l * (1.0 + s) : l + s - l * s;
double p = 2.0 * l - q;

```

```

        r = Hue2Rgb(p, q, h + 1.0 / 3.0);
        g = Hue2Rgb(p, q, h);
        b = Hue2Rgb(p, q, h - 1.0 / 3.0);

        string ToHex(double val)
        {
            int intVal = (int)Math.Round(val * 255.0);
            return intVal.ToString("X2").ToLower();
        }

        return $"#{ToHex(r)}{ToHex(g)}{ToHex(b)}";
    }
}

catch
{
    return hex;
}

return hex;
}

// Operator overloading equivalents (C# supports operator overloading)
public static bool operator >(FormulaTeam a, FormulaTeam b) =>
    ↪ a.ChampionshipPoints > b.ChampionshipPoints;
public static bool operator <(FormulaTeam a, FormulaTeam b) =>
    ↪ a.ChampionshipPoints < b.ChampionshipPoints;

public override string ToString() => GetSummary();
}
}

```

Приложение A2. Класс F1Team.cs

```

using System;

namespace RacingSystem.Core
{
    public class F1Team : FormulaTeam
    {

```

```

public string PowerUnit { get; set; } = "Unknown";
public double BudgetCapMln { get; set; } = 135.0;
public int ConstructorPos { get; set; }

public F1Team() : base() { }

public F1Team(string name, string principal, string hq, string powerUnit)
    : base(name, principal, hq)
{
    PowerUnit = powerUnit;
}

public F1Team(string name, string principal, string hq, string powerUnit,
    ↪ double budget, int ctorPos, int points, int wins, int podiums)
    : base(name, principal, hq, points, wins, podiums)
{
    PowerUnit = powerUnit;
    BudgetCapMln = budget;
    ConstructorPos = ctorPos;
}

public override string GetSeriesName() => "Formula 1";

public override int CalculatePoints(int position)
{
    int[] points = { 25, 18, 15, 12, 10, 8, 6, 4, 2, 1 };
    return (position >= 1 && position <= 10) ? points[position - 1] : 0;
}

public double CalculateBudgetAllocation(double percentage) => BudgetCapMln
    ↪ * (percentage / 100.0);

public string GetPowerUnitStatus()
{
    if (RaceWins > 10) return $"{PowerUnit} [Dominant]";
    if (RaceWins > 4) return $"{PowerUnit} [Competitive]";
    return $"{PowerUnit} [Development]";
}

```

```

[System.Text.Json.Serialization.JsonPropertyName("engine_status")]

```

```

        public string EngineStatus => GetPowerUnitStatus();

        public override string GetSummary()
        {
            return $"{base.GetSummary()} | PU: {PowerUnit} | Budget:
                ↳ ${BudgetCapMln}M";
        }
    }
}

```

Приложение А3. Класс F2Team.cs

```

using System;

namespace RacingSystem.Core
{
    public class F2Team : FormulaTeam
    {
        public string ChassisModel { get; set; } = "Dallara F2 2024";
        public int F1Graduates { get; set; }
        public bool IsFeederSeries { get; set; } = true;

        public F2Team() : base() { }

        public F2Team(string name, string principal, string hq, string chassis)
            : base(name, principal, hq)
        {
            ChassisModel = chassis;
        }

        public F2Team(string name, string principal, string hq, string chassis,
            ↳ int graduates, bool feeder, int points, int wins, int podiums)
            : base(name, principal, hq, points, wins, podiums)
        {
            ChassisModel = chassis;
            F1Graduates = graduates;
            IsFeederSeries = feeder;
        }
    }
}

```

```

public override string GetSeriesName() => "Formula 2";

public override int CalculatePoints(int position)
{
    int[] points = { 15, 12, 10, 8, 6, 5, 4, 3, 2, 1 };
    return (position >= 1 && position <= 10) ? points[position - 1] : 0;
}

public string GetFeederStatus()
{
    if (!IsFeederSeries) return "Independent";
    if (F1Graduates >= 5) return "Elite Feeder (5+ F1 graduates)";
    if (F1Graduates >= 2) return "Active Feeder";
    return "Developing Feeder";
}

[System.Text.Json.Serialization.JsonPropertyName("feeder_status_text")]
public string FeederStatusText => GetFeederStatus();

public override string GetSummary()
{
    return $"{base.GetSummary()} | Chassis: {ChassisModel} | F1 Grads:
    ↪ {F1Graduates}";
}
}
}

```

Приложение А4. Класс FormulaETeam.cs

```

using System;

namespace RacingSystem.Core
{
    public class FormulaETeam : FormulaTeam
    {
        public int SustainabilityScore { get; set; } = 50;
        public double BatteryCapacityKwh { get; set; } = 38.0;
        public string EnergyPartner { get; set; } = "Unknown";
    }
}

```

```

public FormulaETeam() : base() { }

public FormulaETeam(string name, string principal, string hq, string
↪ energyPartner)
    : base(name, principal, hq)
{
    EnergyPartner = energyPartner;
}

public FormulaETeam(string name, string principal, string hq, string
↪ energyPartner, int sustainScore, double batteryKwh, int points, int
↪ wins, int podiums)
    : base(name, principal, hq, points, wins, podiums)
{
    EnergyPartner = energyPartner;
    SustainabilityScore = sustainScore;
    BatteryCapacityKwh = batteryKwh;
}

public override string GetSeriesName() => "Formula E";

public override int CalculatePoints(int position)
{
    int[] points = { 25, 18, 15, 12, 10, 8, 6, 4, 2, 1 };
    return (position >= 1 && position <= 10) ? points[position - 1] : 0;
}

public string GetBatteryEfficiencyRating()
{
    if (BatteryCapacityKwh >= 40.0) return "Next-Gen (40+ kWh)";
    if (BatteryCapacityKwh >= 38.0) return "Gen3 Standard (38 kWh)";
    return "Legacy";
}

[System.Text.Json.Serialization.JsonPropertyName("battery_rating_text")]
public string BatteryRatingText => GetBatteryEfficiencyRating();

public override string GetSummary()
{

```

```

        return $"{base.GetSummary()} | Eco: {SustainabilityScore}/100 |
        ↳ Battery: {BatteryCapacityKwh} kWh";
    }
}
}

```

Приложение А5. Класс FormulaDriver.cs

```

using System;

namespace RacingSystem.Core
{
    public class FormulaDriver
    {
        public int Id { get; set; }
        public int TeamId { get; set; } // Foreign Key to FormulaTeam
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public int RacingNumber { get; set; }
        public int Points { get; set; }

        public FormulaDriver() { FirstName = string.Empty; LastName =
        ↳ string.Empty; }

        public FormulaDriver(string firstName, string lastName, int number, int
        ↳ points = 0)
        {
            FirstName = firstName;
            LastName = lastName;
            RacingNumber = number;
            Points = points;
        }

        public string FullName => $"{FirstName} {LastName}";
        public string DisplayString => $"{FullName} #{RacingNumber}";

        public void AddPoints(int pts) => Points += pts;
    }
}

```